

CI/CD Lab

Overview

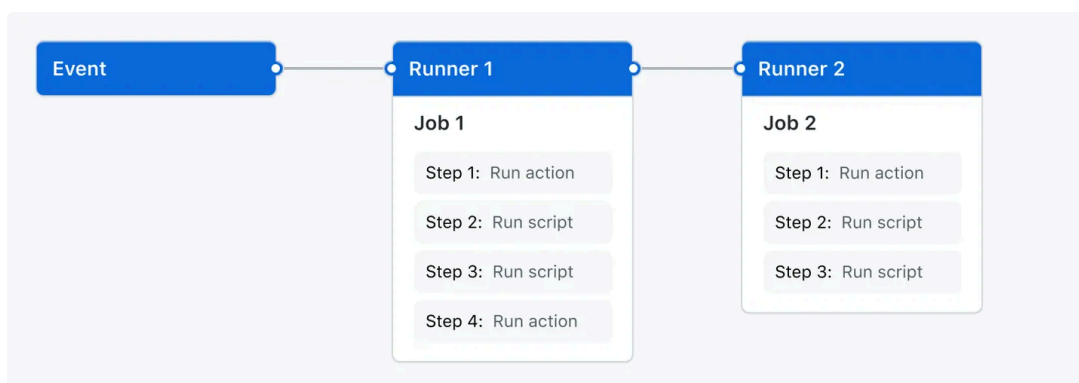
GitHub Actions is a continuous integration and continuous delivery (CI/CD) platform that allows you to automate your build, test, and deployment pipeline. You can create workflows that build and test every pull request to your repository, or deploy merged pull requests to production.

GitHub Actions goes beyond just DevOps and lets you run workflows when other events happen in your repository. For example, you can run a workflow to automatically add the appropriate labels whenever someone creates a new issue in your repository.

GitHub provides Linux, Windows, and macOS virtual machines to run your workflows, or you can host your own self-hosted runners in your own data center or cloud infrastructure.

The components of GitHub Actions

You can configure a GitHub Actions **workflow** to be triggered when an **event** occurs in your repository, such as a pull request being opened or an issue being created. Your workflow contains one or more **jobs** which can run in sequential order or in parallel. Each job will run inside its own virtual machine **runner**, or inside a container, and has one or more **steps** that either run a script that you define or run an **action**, which is a reusable extension that can simplify your workflow.



Workflows

A workflow is a configurable automated process that will run one or more jobs. Workflows are defined by a YAML file checked in to your repository and will run when triggered by an event in your repository, or they can be triggered manually, or at a defined schedule.

Workflows are defined in the `.github/workflows` directory in a repository. A repository can have multiple workflows, each of which can perform a different set of tasks such as:

- Building and testing pull requests
- Deploying your application every time a release is created
- Adding a label whenever a new issue is opened

Events

An event is a specific activity in a repository that triggers a workflow run. For example, an activity can originate from GitHub when someone creates a pull request, opens an issue, or pushes a commit to a repository. You can also trigger a workflow to run on a schedule, by posting to a REST API, or manually.

For a complete list of events that can be used to trigger workflows, see [Events that trigger workflows](#).

Jobs

A **job** is a set of **steps** in a workflow that is executed on the same **runner**. Each step is either a shell script that will be executed, or an **action** that will be run. Steps are executed in order and are dependent on each other. Since each step is executed on the same runner, you can share data from one step to another. For example, you can have a step that builds your application followed by a step that tests the application that was built.

You can configure a job's dependencies with other jobs; by default, jobs have no dependencies and run in parallel. When a job takes a dependency on another job, it waits for the dependent job to complete before running.

For example, you might configure multiple build jobs for different architectures without any job dependencies and a packaging job that depends on those builds.

The build jobs run in parallel, and once they complete successfully, the packaging job runs.

For more information, see [Choosing what your workflow does](#).

Actions

An **action** is a custom application for the GitHub Actions platform that performs a complex but frequently repeated task. Use an action to help reduce the amount of repetitive code that you write in your **workflow** files. An action can pull your Git repository from GitHub, set up the correct toolchain for your build environment, or set up the authentication to your cloud provider.

You can write your own actions, or you can find actions to use in your workflows in the GitHub Marketplace.

Runners

A runner is a server that runs your workflows when they're triggered. Each runner can run a single job at a time. GitHub provides Ubuntu Linux, Microsoft Windows, and macOS runners to run your workflows. Each workflow run executes in a fresh, newly-provisioned virtual machine.

GitHub also offers larger runners, which are available in larger configurations. For more information, see [Using larger runners](#).

If you need a different operating system or require a specific hardware configuration, you can host your own runners.

▼ Exercise 1 Creating your first workflow

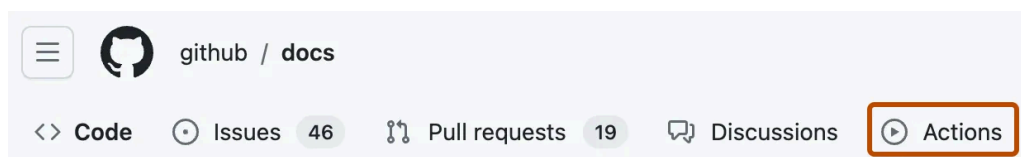
1. In your repository on GitHub, create a workflow file called `github-actions-demo.yml` in the `.github/workflows` directory. To do this:
 - If the `.github/workflows` directory already exists, navigate to that directory on GitHub, click **Add file**, then click **Create new file**, and name the file `github-actions-demo.yml`.
 - If your repository doesn't have a `.github/workflows` directory, go to the main page of the repository on GitHub, click **Add file**, then click **Create new file**, and name the file `.github/workflows/github-actions-demo.yml`. This creates the `.github` and `workflows` directories and the `github-actions-demo.yml` file in a single step.

2. Copy the following YAML contents into the `github-actions-demo.yml` file:

```
name: GitHub Actions Demo
run-name: ${{ github.actor }} is testing out GitHub Actions 🚀
on: [push]
jobs:
  Explore-GitHub-Actions:
    runs-on: ubuntu-latest
    steps:
      - run: echo "🎉 The job was automatically triggered by a ${{ github.event.trigger }} event"
      - run: echo "🐧 This job is now running on a ${{ runner.os }} server hosted by GitHub Actions"
      - run: echo "🔍 The name of your branch is ${{ github.ref }} and your repository is ${{ github.repository }}"
      - name: Check out repository code
        uses: actions/checkout@v4
      - run: echo "💡 The ${{ github.repository }} repository has been cloned to the runner"
      - run: echo "🖨️ The workflow is now ready to test your code on the runner"
      - name: List files in the repository
        run: |
          ls -la ${{ github.workspace }}
      - run: echo "🍏 This job's status is ${{ job.status }}"
```

3. push the file

4. Viewing your workflow results



The example workflow you just added is triggered each time code is pushed to the branch, and shows you how GitHub Actions can work with the contents of your repository.

You can use CI/CD tools to test and deploy your Airflow code. By installing the airflow CLI into your CI/CD process, you can test your DAGs before deploying

them to a production environment.

▼ Exercise 2 Unit testing in airflow

```
# tests/test_dag_integrity.py

from airflow.models import DagBag

def test_dag_integrity():
    dag_bag = DagBag(dag_folder='/home/codespace/airflow/dags', include_examples=False)
    print("DAGs found:", dag_bag.dags.keys())
    print("Import errors:", dag_bag.import_errors)
    assert len(dag_bag.import_errors) == 0, f"DAG import failures: {dag_bag.import_errors}"
    assert 'tweets_classification_pipeline' in dag_bag.dags, "DAG 'tweets_classification_pipeline' not found"
```

- Imports Airflow's `DagBag` class — responsible for parsing and collecting DAGs from a folder.
- Defines a test function to be picked up by `pytest`.
- Creates a `DagBag` object, pointing to your `dags/` directory.
- It scans this folder and tries to load all `.py` files as DAGs.
- `include_examples=False` ensures it ignores Airflow's example DAGs.
- Prints a list of all DAG IDs that were successfully loaded.
- Prints any import errors (e.g., syntax errors, missing imports, etc.)
- Fails the test if any DAG failed to load due to an error (e.g., missing packages, bad code).
- Verifies that a DAG with the ID `'tweets_classification_pipeline'` was found in the scanned files.
- Fails the test if this specific DAG isn't present.

Why This Is Useful:

- Catches broken DAGs early (syntax/import errors)
- Ensures your DAG files are named and structured correctly

- Verifies that specific critical DAGs are present
- Lightweight — great for CI/CD (GitHub Actions, GitLab CI, etc.)

▼ Exercise 3 Integrate unit testing with CI workflow

.github/workflows/ci.yml

```
name: Airflow DAG Test

on:
  push:
    branches: [main, airflow]
  pull_request:

jobs:
  dag-test:
    runs-on: ubuntu-latest
    container:
      image: python:3.12-bullseye
      options: --entrypoint /bin/bash
    env:
      AIRFLOW_HOME: /github/home/airflow
      ROOT_DIR: ${GITHUB_WORKSPACE}

    steps:
      - name: Checkout repo
        uses: actions/checkout@v3

      - name: Show environment info
        run: |
          echo "ROOT_DIR=$ROOT_DIR"
          ldd --version | head -n1
          python --version

      - name: Install dependencies
```

```
run: |
    apt-get update && apt-get install -y build-essential
    pip install --upgrade pip
    pip install "apache-airflow[celery]==2.10.5" --constraint "https://raw.git
    pip install -r requirements.txt

- name: Initialize Airflow database
  run: airflow db init

- name: Pre testing dags
  run: |
    echo "Current directory:"
    pwd
    python dags/tweets_etl.py
    echo "Listing Airflow DAGs:"
    airflow dags list

- name: Run DAG integrity test
  run: pytest tests/test_dag_integrity.py
```

dags/tweets_etl.py

```
import json
import pendulum
from airflow.decorators import dag, task
import pandas as pd
from langchain_ollama.llms import OllamaLLM
from langchain_core.prompts import ChatPromptTemplate
from pydantic import BaseModel, Field
from langchain_core.output_parsers import PydanticOutputParser
from langchain_ollama import ChatOllama

class Sentiment(BaseModel):
```

```

sentiment: str = Field(description="tweet sentiment")

@dag(
    dag_id="tweets_classification_pipeline",
    schedule=None,
    start_date=pendulum.datetime(2021, 1, 1, tz="UTC"),
    catchup=False,
    tags=["example"],
)
def tweets_classification_pipeline():

    @task()
    def extract():

        """
        ##### Extract task
        """

        splits = {
            'train': 'sentiment/train-00000-of-00001.parquet',
            'test': 'sentiment/test-00000-of-00001.parquet',
            'validation': 'sentiment/validation-00000-of-00001.parquet'
        }

        df = pd.read_parquet(
            "hf://datasets/cardiffnlp/tweet_eval/" + splits["train"],
            engine="pyarrow",
            columns=None, # optional: limit to specific columns
            filters=None, # optional: row filtering (not for head)
        )

        df = df.head(5)
        return df[['text']]

    @task(multiple_outputs=True)
    def transform(tweets_df: pd.DataFrame):

```



```

"""
#### Transform task
Classifies the sentiment of tweets using Ollama.
"""

# Define the sentiment classification prompt
template = """Classify the sentiment of the following tweet as Positive, N

Tweet: {tweet}

Sentiment:"""
prompt = ChatPromptTemplate.from_template(template)

llm = ChatOllama(model="smollm")
structured_llm = llm.with_structured_output(Sentiment)
# Classify sentiment for each tweet
sentiments = []
for tweet in tweets_df['text']:
    chain = prompt | structured_llm
    response = chain.invoke({"tweet": tweet})
    sentiments.append(response.sentiment)

# Add sentiment classification to the DataFrame
tweets_df['sentiment'] = sentiments
return {"classified_tweets": tweets_df.to_dict(orient="records")}

@task()
def load(classified_tweets: list):
    """
    #### Load task
    Prints only the first classified tweet with its sentiment and saves all to a J
    """

    # Define the output file path
    output_file = "/workspaces/depi-eyouth/output/classified_tweets.json"

    # Save the classified tweets to a JSON file
    with open(output_file, "w") as f:

```

```

        json.dump(classified_tweets, f, indent=4)

    print(f"Classified tweets saved to {output_file}")

    # Print only the first tweet with its sentiment
    if classified_tweets:
        first_tweet = classified_tweets[0]
        print(f"First Tweet: {first_tweet['text']], Sentiment: {first_tweet['sentim

tweets_df = extract()
classified_tweets = transform(tweets_df)
load(classified_tweets["classified_tweets"])

tweets_classification_pipeline()

```

tests/test_dag_integrity.py

```

# tests/test_dag_integrity.py

from airflow.models import DagBag
import os

def test_dag_integrity():
    root_dir = os.environ.get("ROOT_DIR", ".")
    dags_path = os.path.join(root_dir, "dags")
    dag_bag = DagBag(dag_folder=dags_path, include_examples=False)
    print("DAGs found:", dag_bag.dags.keys())
    print("Import errors:", dag_bag.import_errors)
    assert len(dag_bag.import_errors) == 0, f"DAG import failures: {dag_bag.im
    assert 'tweets_classification_pipeline' in dag_bag.dags, "DAG 'tweets_class

```